DBMS – Lab
BSCS [-A]

FINAL TERM PROJECT PROPOSAL

Submitted By [Group No. DB 01]:

- Mian AbdurRahman Qayyum (03-134242-039)
- Haider Ali (03-134242-022)
- Ahmad Uneeb (03-134242-047)

Submitted to: Miss Nadia Shakir

PROJECT NAME
Food & Catering Order Manager

DEPARTMENT OF COMPUTER SCIENCES
BAHRIA UNIVERSITY, LAHORE CAMPUS

Table of Contents

Marks Distribution 3
Abstract 4
1. Introduction 5
2. Objectives 5
3. Database Design 6
4. System Features 7
5. Technologies Used 8

Marks Distribution

---------- *For Instructor Use Only* ----------

Project Grading Sheet				
Sr. No.	**Task**	**Max Marks**	**Obtained Marks**	**Comments (if any)**
1.	Report / Documentation	2		
2.	Database Design & Schema	5		
3.	Implementation / Working System	10		
4.	Viva (Oral / Task Based or Both)	8		
Total		**25**		Signature: _____

Abstract

The Food & Catering Order Manager is a full-stack web application developed using Python Flask, MySQL, and Bootstrap 5, designed to digitize and streamline operations for catering and food preparation businesses. It replaces inefficient paper-based and spreadsheet workflows with a centralized, role-aware platform that manages the complete order lifecycle — from customer intake to kitchen preparation and final delivery.

The system provides three distinct operational views — an admin dashboard, a kitchen display, and a delivery interface — each tailored to the needs of a specific staff role. Real-time synchronization via Flask-SocketIO ensures that status changes made by one user are immediately reflected across all connected devices, enabling seamless coordination between front-of-house staff, kitchen teams, and delivery personnel.

Key capabilities include multi-item order entry with live cost calculation, a five-stage status pipeline (Received → Logged → In Preparation → Ready → Delivered), a printable kitchen pick list, overdue order alerts, a live activity dashboard with Chart.js visualizations, and an internal comments system per order. The application is deployable on PythonAnywhere at no cost, making it accessible to small and medium food businesses.

1. Introduction

Small and medium-sized catering businesses face a persistent operational challenge: managing high volumes of complex food orders across multiple preparation stages using only paper records or basic spreadsheets. This approach leads to missed orders, preparation errors, poor staff coordination, and no real-time visibility into order status.

Project Motivation

The Food & Catering Order Manager was designed to solve these pain points by providing a purpose-built digital system that handles the full order workflow. By applying core DBMS concepts — relational schema design, referential integrity, transactional queries, and role-based data access — the project demonstrates how database-driven applications deliver measurable real-world value.

The system addresses these specific problems:

- No centralized record of orders — staff rely on handwritten notes
- Kitchen team has no live view of what needs to be prepared
- Delivery staff do not know which orders are ready to dispatch
- Managers have no visibility into overdue or at-risk orders
- Re-ordering for repeat customers requires re-entering all details manually

2. Objectives

The system delivers the following functional modules:

1. Secure Role-Based Login

- Session-based authentication using Werkzeug password hashing
- Three staff roles: Admin, Kitchen, Delivery
- All routes protected by a login_required decorator

2. Customer Management

- Add, view, and search customers
- Full order history per customer
- One-click re-order from any past order with pre-filled form

3. Order Entry & Management

- Multi-item order form with dynamic row add/remove via vanilla JavaScript
- Live running total that updates as items and quantities change
- Search and filter orders by customer name, date, and status
- Overdue order highlighting in the order list

4. Five-Stage Status Pipeline

- Received → Logged → In Preparation → Ready → Delivered
- Forward-only validation — status cannot go backwards
- One-click status buttons on both order list and order detail pages
- Visual stepper on order detail showing completed, current, and upcoming stages

5. Kitchen View

- Dedicated full-width display for kitchen staff showing only active orders
- Auto-refreshes every 60 seconds with last-updated timestamp
- DUE TODAY and OVERDUE badges for time-sensitive orders
- Start Preparation and Mark Ready buttons per order card

6. Delivery View

- Shows only orders in Ready status with customer address and contact
- Full-width Confirm Delivered button per order

7. Real-Time Sync & Notifications

- Flask-SocketIO broadcasts status changes to all connected devices instantly
- Floating Bootstrap toast notifications appear on every connected tab
- Navbar badge counters update live without page reload
- Overdue banner checks every 5 minutes and alerts staff automatically

8. Dashboard & Reporting

- Metric cards: orders today, in kitchen, ready, delivered, overdue

- Live activity feed showing last 10 status changes
- Chart.js donut chart showing status distribution updated via SocketIO
- Daily summary page grouping orders by status
- Printable kitchen pick list grouped by product with total quantities

9. Internal Comments System

- Staff can add timestamped notes to any order
- Comments stored in order_comments table linked to both order and staff
- New comments appear on all open detail tabs in real time via SocketIO

3. Database Design

The MySQL database catering_db is structured around six relational tables:

Table	Key Columns	Purpose
customers	id, name, phone, email, address	Stores client information for order association and history
products	id, name, unit, price_per_unit, is_active	Product catalog with unit type and pricing for order items
staff	id, name, email, password_hash, role	Authenticated users with role-based access (admin/kitchen/delivery)
orders	id, customer_id, staff_id, delivery_date, status, notes	Master order record linking customer, staff, and lifecycle status
order_items	id, order_id, product_id, quantity, unit_price	Line items per order with price snapshot at time of order entry
order_comments	id, order_id, staff_id, comment, created_at	Internal timestamped notes attached to orders by staff members

Relationships & Constraints

- orders.customer_id → customers.id (each order belongs to one customer)
- orders.staff_id → staff.id (each order created by one staff member)
- order_items.order_id → orders.id ON DELETE CASCADE (items deleted with order)
- order_items.product_id → products.id (price snapshot preserved at entry time)
- order_comments.order_id → orders.id ON DELETE CASCADE
- order_comments.staff_id → staff.id (comment author tracked)
- orders.status uses ENUM enforcing the five-stage pipeline at the DB level

4. System Features & Pages

The application contains the following URLs and corresponding templates:

Route	Template	Description
/dashboard	dashboard.html	Live metric cards, activity feed, status donut chart
/orders	orders.html	Full order list with search, filter, and one-click status buttons
/orders/new	new_order.html	Dynamic multi-item order form with live total calculation
/orders/<id>	order_detail.html	Order detail with pipeline stepper, items, comments
/kitchen	kitchen.html	Kitchen-only view, auto-refresh, preparation action buttons
/delivery	delivery.html	Ready orders with customer address and confirm delivery button
/customers	customers.html	Customer list, add modal, order history and re-order
/products	products.html	Product catalog with add/edit modal
/summary	summary.html	Daily summary grouped by status with metric cards
/picklist	picklist.html	Printable prep list grouped by product across all active orders

5. Technologies Used

Core Technologies

- Python 3.10 — Backend application language
- Flask — Lightweight WSGI web framework for routing and templating
- Flask-SocketIO + Eventlet — Real-time WebSocket communication across all clients
- PyMySQL — MySQL database connector for Python
- Werkzeug — Password hashing and security utilities
- MySQL — Relational database management system via XAMPP locally
- Bootstrap 5.3 — Responsive frontend UI framework (loaded via CDN)
- Bootstrap Icons — Icon library for navigation and UI elements
- Chart.js — Client-side donut chart for dashboard status visualization
- Jinja2 — Server-side HTML templating engine (built into Flask)
- Vanilla JavaScript — Dynamic form rows, live total calculation, SocketIO client

Development & Deployment Tools

- XAMPP — Local development server providing Apache and MySQL
- phpMyAdmin — GUI for MySQL database setup and management
- GitHub Student Developer Pack — Free access to deployment and dev tools
- PythonAnywhere — Free cloud hosting platform for Python/Flask applications
- Visual Studio Code — Primary code editor
- Git — Version control for source code management

DBMS Concepts Applied

- Relational schema design with primary and foreign key constraints
- Referential integrity with ON DELETE CASCADE for order items and comments
- ENUM data type for enforcing valid status values at the database level
- JOIN queries across multiple tables for order detail and pick list generation
- Aggregate functions (COUNT, SUM) for dashboard metrics and daily summaries
- Parameterized queries for SQL injection prevention
- Price snapshotting — unit_price stored at order time, not looked up dynamically
- Role-based data access enforced at both application and session level